



## PROJECT AND TEAM INFORMATION

### Project Title

**WALnut: A Concurrent Auction Engine with Deadlock Detection & Recovery**

### Student / Team Information

|                                            |                                                                                                                                                                     |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><i>Team Name:</i><br/><i>Team #</i></p> | <p><i>The Acid Mechanics</i><br/><i>T038</i></p>                                                                                                                    |
| <p><b>Team member 1 (Team Lead)</b></p>    | <p><i>Suyal, Vinayak – 240211122</i><br/><i>suyalvinayak22@gmail.com</i></p>    |
| <p><b>Team member 2</b></p>                | <p><i>Singh Rawat, Sushmit – 24021051</i><br/><i>7838sushmit@gmail.com</i></p>  |
| <p><b>Team member 3</b></p>                | <p><i>Rawat, Kanak – 240211907</i><br/><i>kavyarawatkavya831@gmail.com</i></p>  |
| <p><b>Team member 4</b></p>                | <p><i>Jugran, Suhavi – 240211640</i><br/><i>suhavijugran@gmail.com</i></p>      |

# PROPOSAL DESCRIPTION

## Motivation

Modern applications rely on databases to store and process large volumes of data safely, even when many users act on that data at the same time. This project is motivated by a live auction, where multiple bidders compete for the same item and the system must guarantee that every accepted bid is correct, that no two bids can both be recorded as the winning bid, and that a crash never loses or duplicates a committed bid.

To address this, **WALnut** deliberately combines Operating Systems concepts (thread pools, synchronization primitives, buffer management, deadlock handling) with core DBMS concepts (schema design, indexing, transaction management, concurrency control, and recovery) around one small, observable workload.

## State of the Art / Current solution

Production database systems such as PostgreSQL and SQLite already solve this class of problem. SQLite uses a page-based architecture with a page cache, locking, and B-tree storage, and its WAL mode logs changes to a separate file that is later incorporated into the database at checkpoints. PostgreSQL similarly records changes in a write-ahead log before the corresponding data page is written back, enabling crash recovery. Both systems use wait-for-graph deadlock detection rather than a resource-declaration scheme such as Banker's Algorithm, because a general transaction may request locks dynamically.

These production engines are highly complex and abstracted away from users. WALnut is intentionally scoped as a small educational storage and transaction engine—not a general-purpose DBMS—built around a fixed, observable workload so that locking, storage, buffering, deadlock handling, and crash recovery can be implemented and demonstrated within eight weeks.

## Project Goals and Milestones

### Project Objectives

- Build a persistent, page-based auction data engine supporting a fixed set of parameterized operations instead of general-purpose SQL.
- Guarantee safe multithreaded concurrency: simultaneous bids on the same item must never corrupt or duplicate auction state.
- Implement a custom storage manager and buffer pool for file, page, and memory management.
- Detect and resolve deadlocks using a wait-for graph instead of a resource-declaration scheme.
- Protect committed data through Write-Ahead Logging, checkpointing, and crash recovery.
- Provide a live dashboard where transactions, locks, the wait-for graph, WAL activity, and engine performance can be observed.

### Key Milestones

- **Week 1 (Design Freeze):** Finalize the fixed auction schema (Users, Items, Bids), ER diagram, functional dependencies, fixed operation API, and architecture.
- **Week 2 (Storage Foundation):** Implement the on-disk page format, file manager, record layout, and database initialization.
- **Week 3 (Buffer & Indexing):** Implement the buffer pool (pin/unpin, dirty pages, Clock/LRU replacement) and a hash index on `item_id`.
- **Week 4 (Single-threaded Operations):** Wire the fixed auction operations end-to-end without concurrency.
- **Week 5 (Concurrency Core – MVP):** Introduce a POSIX thread pool and a lock manager. We will implement Strict 2PL to make `placeBid` operations fully atomic.
- **Week 6 (Deadlock Handling):** Develop a wait-for graph to spot cycles and select victims for abort/retry logic. We'll test this out by running overlapping `reserveBundle` queries that grab identical items in mixed orders.
- **Week 7 (Recovery):** Set up a Write-Ahead Log to record before/after images and transaction commits. We will run fault-injection tests to prove the engine can successfully redo finished operations and rollback incomplete ones.
- **Week 8 (Integration & Demo):** Connect the embedded REST API to a web dashboard. Finish up load benchmarking, write the final documentation, and prep the demonstration script.

## Project Approach

### Schema Design & Fixed API:

We are giving the engine a strict relational schema (Users, Items, Bids) and predefined functions like `placeBid` or `createUser`. These directly execute core relational-algebra steps. To deliberately create scenarios where deadlocks happen, we added a `reserveBundle` operation. This lets a user try to bid on a group of related items on an all-or-nothing basis, forcing the database to request several locks at once.

### Transaction & Concurrency Layer:

Every `placeBid` request runs as a standard ACID transaction. We manage concurrent traffic using a POSIX thread pool paired with mutexes and condition variables. To avoid dirty writes or inconsistent reads, a Strict 2PL lock manager grabs exclusive locks and holds them until a commit or rollback happens. When executing a `reserveBundle`, the system secures all necessary item locks before touching any data.

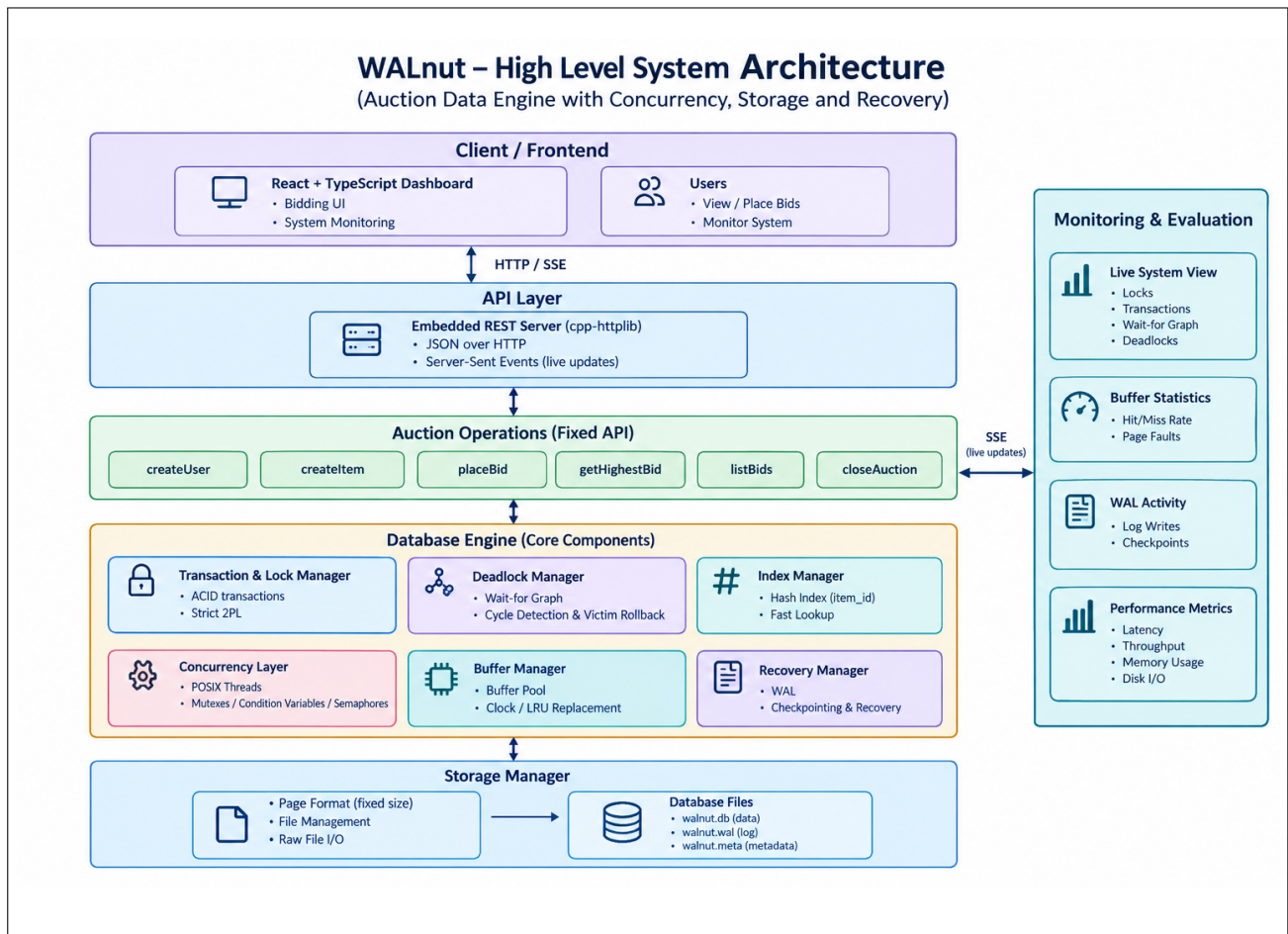
### Deadlock Detection:

We maintain a wait-for graph to monitor who is waiting on whom. If a cycle pops up—say, Transaction A has Lock 1 but needs Lock 2, while Transaction B has Lock 2 and needs Lock 1—a background thread steps in. It picks a victim, rolls it back, frees up those held locks, and updates the graph so the other transactions can move forward. The aborted victim then gets retried automatically.

### Storage, Recovery & Interface:

For persistent storage, a custom manager reads and writes fixed-size pages to disk, caching them in memory with a buffer pool driven by a Clock/LRU policy. We also use a simple hash index to speed up `item_id` searches. Before dumping a modified page to disk, its updates get written into the Write-Ahead Log. Each log entry keeps the old value for undo purposes and the new value for redo. During a restart, finished transactions get redone, while abandoned ones are wiped. A lightweight REST server (cpp-httpplib) runs inside the C++ process, sending live telemetry to a React dashboard via Server-Sent Events.

## System Architecture (High Level Diagram)



## Project Outcome / Deliverables

- A persistent storage engine relying on a custom page-based disk format rather than third-party databases.
- A dedicated buffer pool featuring a Clock/LRU page replacement strategy.
- Pre-defined auction queries designed to run securely as ACID transactions.
- A Strict 2PL lock manager built to stop simultaneous bids from messing up database state.
- A background deadlock detector that spots wait-for cycles, aborts a victim, and retries the process (specifically tested via the **reserveBundle** feature).
- Crash recovery mechanics using a Write-Ahead Log to safely restore data after a forced crash.
- A monitoring dashboard displaying live metrics like active locks, the wait-for graph, buffer pool cache rates, and WAL writes.
- Benchmark data showing how the system performs under heavy bidding, tracking metrics like latency, overall throughput, and deadlock frequency.
- Full project documentation detailing the underlying architecture, table schemas, and testing outcomes.

## Assumptions

- The engine runs entirely as a single instance. We are not designing this to scale out across a distributed cluster or multiple separate machines.
- `placeBid` operates on one item, while `reserveBundle` is the multi-item atomic operation used to exercise wait-for cycles.
- The number of simultaneous simulated bidders remains within the capacity of a single development and demonstration machine.
- Authentication and authorization are out of scope; users are identified by a fixed `user_id` for demonstration purposes.
- Catastrophic hardware failure and disk corruption are out of scope; recovery targets process crashes and abrupt restarts.
- The crash-recovery demonstration uses controlled fault injection at transaction and page-flush boundaries.

## References

1. SQLite Documentation, “Architecture of SQLite.” <https://www.sqlite.org/arch.html>
2. PostgreSQL Documentation, “Write-Ahead Logging (WAL).” <https://www.postgresql.org/docs/current/wal-intro.html>
3. MySQL Documentation, “InnoDB Locking and Transaction Model.” <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-transaction-model.html>
4. IBM Documentation, “POSIX Thread Programming.” <https://www.ibm.com/docs/en/zos/2.5.0?topic=services-thread-programming>
5. Silberschatz, Korth, Sudarshan. *Database System Concepts* – Concurrency Control and Deadlock Handling.
6. Silberschatz, Galvin, Gagne. *Operating System Concepts* – Deadlocks: Detection and Recovery.